Reflections on Trusting Trust

Why you can’t fully trust tooling

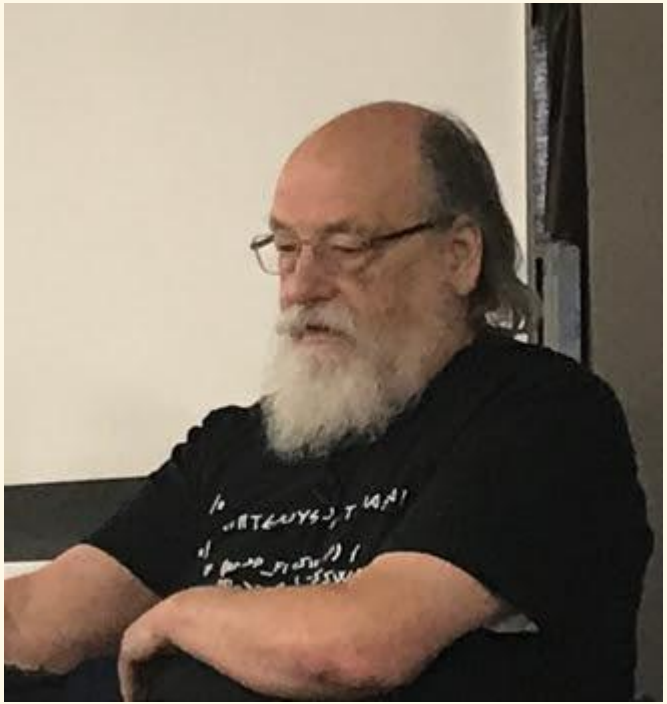
Rithvik R

If you read every line of a program’s source code and it’s clean, can you trust it?

(spoiler: no)

Ken Thompson

- Co-created C and UNIX
- Awarded the 1983 Turing Award (with Dennis Ritchie)
- It’s basically the Nobel Prize of computer science!



Stage 1 - Quines

A quine is basically a non-empty program that gives its own source code as its only output.

A simple quine in C (thanks stackoverflow):

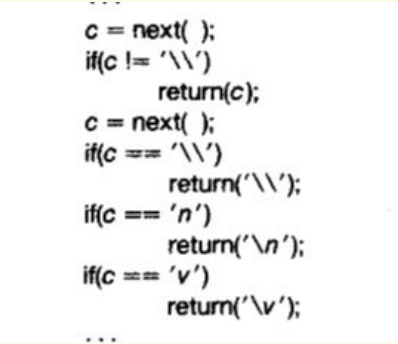
```
#include <stdio.h>
char *s="#include <stdio.h>%cchar *s=%c%s%c;%cint main(){printf(s, 10, 34, s, 34, 10, 10);}%c";
int main(){printf(s, 10, 34, s, 34, 10, 10);}
```

What this tells us:

- A quine can be written by another program.
- Whatever we specify will be reproduced (yes, even comments).

Stage 2 - Bootstrapping

- To edit a compiler's behaviour, you modify its source code and compile it using the old binary
- This is called "bootstrapping"
- What if the required changes use some syntax that the old compiler doesn't know about? Compiler will return an error!



```
...
c = next( );
if(c != '\\')
        return(c);
c = next( );
if(c == '\\')
        return('\\');
if(c == 'n')
        return('\n');
if(c == 'v')
        return('\v');
...
```

\v - invalid syntax (it's a new feature)
compiler will error out

We need to “teach” the compiler about this new syntax. How?

- Hardcode the intended behaviour and compile to get a binary
- Make the required changes and compile using the above binary
    - this will compile because the binary has the hardcoded mapping
- Now any future builds will carry on the behaviour

The compiler has “learned” the new syntax now!

```
c = next( );
if(c != '\\')
        return(c);
c = next( );
if(c == '\\')
        return('\\');
if(c == 'n')
        return('\n');
if(c == 'v')
        return(11);
...
```

hardcoded \v to return 11
mapping is hardcoded, used to compile new version

Stage 3 - Combining the two stages

How do we add a backdoor to a program?

- The compiler can be made to recognize when it’s compiling a specific program.
    - If we recognize that we’re compiling the target program, we can modify the compiler such that it deliberately miscompiles.
- But this would be too easy to spot, since the bugged code is right there in the source!

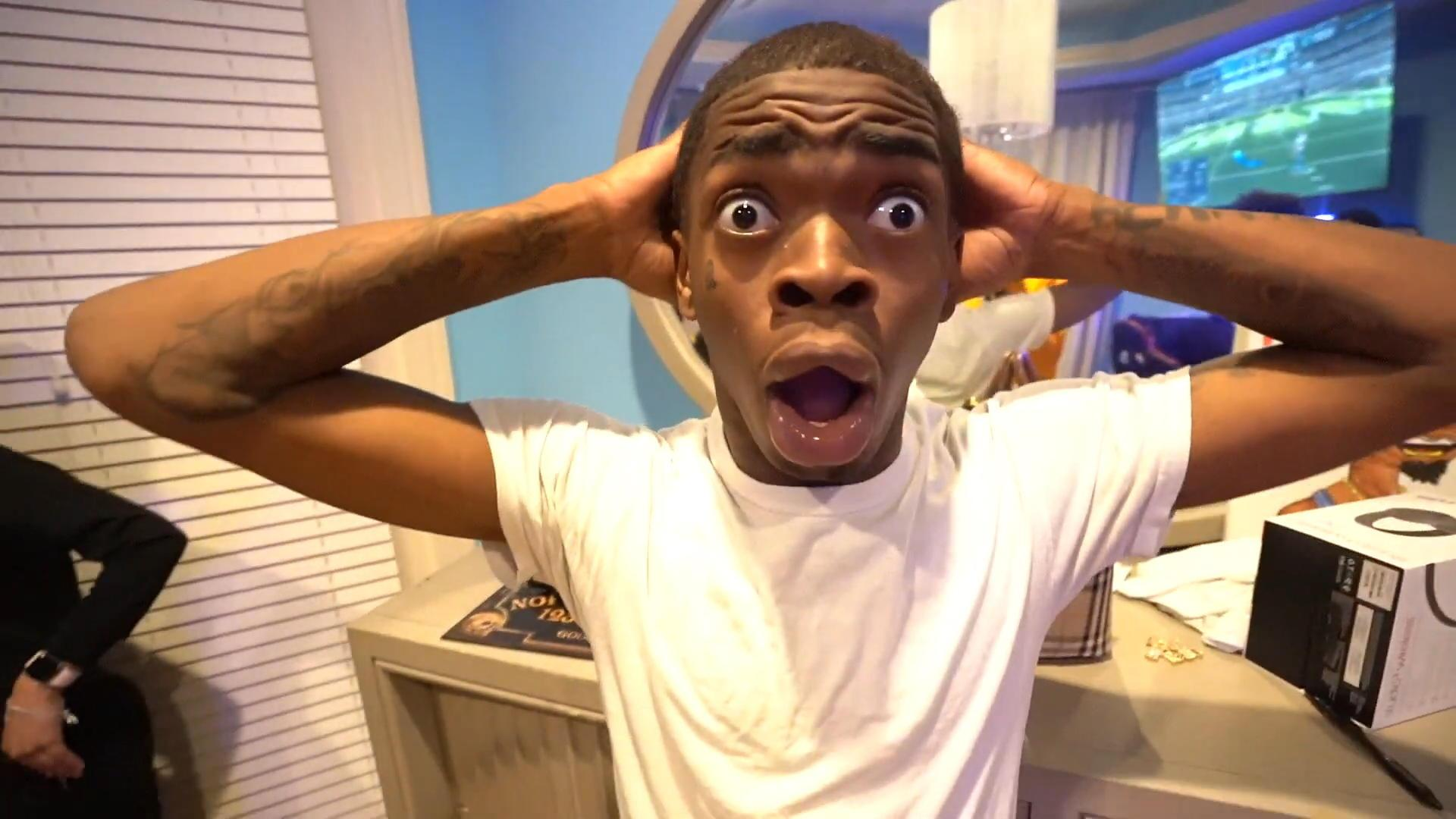
- The trick is to add a second Trojan horse that checks whether the compiler is compiling itself.
    - If so, we reinsert both the above Trojan horses into the compiler code
- Then, we compile this modified code using a bug-free compiler to generate the bugged binary
- Now we remove the bug code from the source.
- Whenever we compile with the bugged binary, the bugs are reinserted, even though there is no code for it in the source files!

Bugged compiler psuedocode

```
compile(src) {
    if (match(src, “target”)) {
        emit(BACKDOOR_OBJECT_CODE);
        return;
    }
    if (match(src, ”compiler”)) {
        emit(SELF_OBJECT_CODE);
        return;
    }
    compile_normal(src);
}
```

Compiled version of this whole function

You can’t trust code that you didn’t write yourself.
Source inspection alone cannot guarantee safety.



How do I detect such an attack?

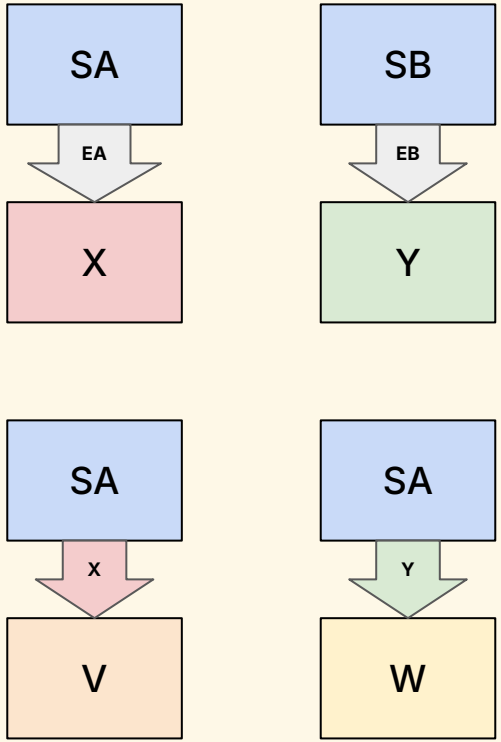
This technique is called Diverse Double-Compiling.

Suppose we have two *independent* compilers A & B.
Source code of A - SA, source code of B - SB
Executable binary of A - EA, binary of B - EB
To check if compiler A is bugged:

- Compile SA with EA, giving binary X
- Compile SB with EB, giving binary Y

X and Y should be functionally equivalent.

- Compile SA with X and Y to get binaries V and W
- If A isn't bugged, V and W should be bit-for-bit equivalent.



Thank you / questions?